ETC: Exact Topological Computing

User Manual and API Reference

Maitri Aniruddhbhai Savaliya
GSFC University, Vadodara, Gujarat, India
maitrisavaliya05@gmail.com

This document is the machine-readable user manual for the ETC Python library, submitted as supplementary material accompanying the ACM TOMS manuscript *ETC: Exact Topological Computing*. It describes all public constructors, methods, and parameters across the ten public ETC modules.

Requirements: Python >= 3.9, sympy >= 1.12, mpmath >= 1.3.0. Lean 4 is required only for proof checking; it is not a Python package and must be installed separately (see INSTALL).

Package structure

Module	Contents
etc.core	ExactReal, Interval, ExactComplex, ExactInteger
etc.geometry	AlgebraicCurve, ParametricCurve, AlgebraicSurface, ExactPolygon
etc.topology	Space, Path, Homotopy, winding_number, Manifold, Chart, Atlas
etc.analysis	Calculus, PowerSeries, TaylorSeries, ODE
etc.verify	Certificate, CertificateStore, Lean4Exporter, lean_syntax, symbolic
etc.problems	Problem, ProblemRegistry, canonical_problems (RumpPolynomial, L5WindingNumber)
etc.certified	Certified[V] generic wrapper

The on-disk package layout is: `etc/` (library code, seven sub-packages plus `certified.py`), `tests/` (`test_core.py`, `test_problems.py`, `conftest.py`, and a `portability/` directory of saved cross-platform run transcripts that are not pytest-collected), `scripts/` (`run_comparison.py` and `benchmark.py`), and `doc/` (this manual, pre-built and shipped with the package).

2. etc.core.real -- ExactReal

ExactReal represents a real number x as a lazy Cauchy sequence over Q: a function `seq: int -> Fraction` such that |seq(n) - x| < 2^{-n} for all n >= 0. Results are memoised by precision.

2.1 Constructors

Constructor	Description
ExactReal(seq)	From a raw SeqFn: int -> Fraction
ExactReal.from_rational(p, q=1)	Exact rational p/q. p may be int or Fraction
ExactReal.from_int(n)	Integer n (shorthand for from_rational(n))
ExactReal.zero()	The constant 0
ExactReal.one()	The constant 1
ExactReal.pi()	pi via Machin's formula. O(n^2 log n)
ExactReal.e()	e = sum 1/k!. O(n^2 log n)
ExactReal.log2()	ln(2). O(n^2 log n)
ExactReal.sqrt2()	sqrt(2) via Newton's method. O(n^2 log^2 n)
ExactReal.from_decimal(s)	Parse decimal string e.g. '3.14159' exactly
ExactReal.from_sympy(expr, sym, val)	Substitute val (ExactReal) into sympy expr

2.2 Evaluation

Method	Description
eval(n: int) -> Fraction	Return r in Q with |r - x| < 2^-n. Memoised: calling twice computes once. n >= 0 required.
timed_eval(n: int) -> (Fraction, float)	Evaluate and return (result, wall_seconds). Returns (cached, 0.0) on cache hit.
to_decimal(digits=20) -> str	Return decimal string with 'digits' significant figures.

2.3 Arithmetic

Method	Returns	Notes
add(other)	ExactReal	self + other
sub(other)	ExactReal	self - other
mul(other)	ExactReal	self * other; extra bits computed once at construction
div(other)	ExactReal	self / other; requires other != 0
neg()	ExactReal	-self
abs_val()	ExactReal	|self|
recip()	ExactReal	1/self; raises ZeroDivisionError if self is zero
pow_int(k: int)	ExactReal	self^k via binary exponentiation; k >= 0
pow_rational(p, q)	ExactReal	self^(p/q); dispatches to pow_int + Newton root
sqrt()	ExactReal	sqrt(self) via Newton. O(n^2 log^2 n)
cbrt()	ExactReal	cbrt(self) via Newton. O(n^2 log^2 n)

2.4 Transcendental functions

Method	Description
exp()	e^self with argument reduction; O(n^2 log n)
log()	ln(self) for self > 0; reduces to [1/2, 2) then arctanh series
sin()	sin(self); full argument reduction to [-pi/2, pi/2] then Taylor
cos()	cos(self); argument reduction to [-pi, pi] then Taylor
tan()	tan(self) = sin/cos
atan()	arctan(self); Gregory series with reduction to [0, 1/2]

2.5 Comparison

Method	Description
lt(other, prec=53) -> bool	self < other at given precision; raises ValueError if indistinguishable
gt(other, prec=53) -> bool	self > other
eq_approx(other, prec=53) -> bool	True if |self - other| < 2^-prec at prec bits
sign(prec=53) -> int	Quick sign in {-1, 0, 1} at prec bits (not certified; use etc.sign() for certified result)

2.6 Python operator overloads

ExactReal supports +, -, *, / with other ExactReal instances or with int/Fraction. float(x) calls x.eval(53) and converts to Python float. repr(x) displays the 53-bit float approximation.

2.7 Example

```
from etc import ExactReal
from fractions import Fraction

# Exact rational constant
a = ExactReal.from_rational(Fraction(1, 3))

# pi to 128 bits
pi_128 = ExactReal.pi().eval(128)   # returns exact Fraction

# Composed expression
x = a.add(ExactReal.pi()).mul(ExactReal.sqrt2())
print(x.to_decimal(30))   # 30-significant-digit decimal string

# Rump's polynomial (exact)
a_ = ExactReal.from_rational(77617)
b_ = ExactReal.from_rational(33096)
b2 = b_.mul(b_); b4 = b2.mul(b2); b6 = b4.mul(b2); b8 = b4.mul(b4)
a2 = a_.mul(a_)
result = (
    ExactReal.from_rational(Fraction(33375, 100)).mul(b6)
    .add(a2.mul(
        ExactReal.from_rational(11).mul(a2).mul(b2)
        .sub(b6).sub(ExactReal.from_rational(121).mul(b4))
        .sub(ExactReal.from_rational(2))
    ))
    .add(ExactReal.from_rational(Fraction(11, 2)).mul(b8))
    .add(a_.div(b_.mul(ExactReal.from_rational(2))))
)
print(result.eval(80))   # Fraction(-54767, 66192)
```

3. etc.core.sign -- sign()

sign() computes the sign of an ExactReal with a machine-checkable certificate. It evaluates the Cauchy sequence at increasing precision until the sign is unambiguous, then returns a Certified[int] with value in {-1, 0, +1}.

Signature

```
sign(x: ExactReal, max_prec: int = 128) -> Certified[int]
```

Parameters

Parameter	Type	Description
x	ExactReal	The number whose sign is to be determined
max_prec	int	Maximum precision bits to try (default 128). If the sign cannot be determined at this precision, 0 is returned with a note.

Returns

Certified[int] with .value in {-1, 0, +1}. The .proof attribute is a Certificate recording the precision at which certainty was achieved and the approximant used.

Algorithm

Evaluate x.eval(n) for n = 8, 16, 32, ..., max_prec. At each precision n, if |approx| > 2^-(n-1), the sign is certain. If no precision settles it, return 0. Termination guarantee: for x != 0 with |x| >= 2^-k, the algorithm terminates at prec = max(2k+4, 8).

Example

```
from etc import ExactReal, sign

x = ExactReal.pi().sub(ExactReal.from_rational(3))
result = sign(x)
print(result.value)             # 1
print(result.is_valid())        # True
print(result.proof.summary())   # Certificate [VALID]
                                 #  Claim  : sign(x) = +1
                                 #  Method : cauchy_evaluation(prec=8)

# Serialise to JSON
import json
d = json.loads(result.proof.to_json())
print(d['evidence']['prec_bits'])   # 8
```

4. etc.core.interval -- Interval

Interval is a closed interval [lo, hi] of exact rational endpoints, the fast inner layer used by space predicates and path checking. Every operation produces an interval guaranteed to contain the true result -- no false negatives, possible false positives (conservatism). An interval [lo, hi] represents the set { x in R | lo <= x <= hi }; its width w = hi - lo measures over-approximation. All internal bounds are kept as Fraction (exact).

4.1 Constructors

Constructor	Description
Interval(lo, hi)	From two Fraction bounds. Raises ValueError if lo > hi.
Interval.from_real(x, prec=40)	Build [x-eps, x+eps] where eps = 2^-prec, from an ExactReal x
Interval.from_rationals(a, b)	From two Fraction/int endpoints (order-independent)
Interval.point(x)	Degenerate interval [x, x]

4.2 Properties and predicates

Method	Description
width() -> Fraction	hi - lo
midpoint() -> Fraction	(lo + hi) / 2
contains_zero() -> bool	True if 0 in [lo, hi]
contains(v) -> bool	True if lo <= v <= hi
is_positive() -> bool	Certainly positive: lo > 0
is_negative() -> bool	Certainly negative: hi < 0
certainly_lt(other) -> bool	True if every element of self is < every element of other (self.hi < other.lo)
certainly_gt(other) -> bool	other.certainly_lt(self)
certainly_le(other) -> bool	self.hi <= other.lo
certainly_eq(other) -> bool	Only certain if both are point intervals with the same value
possibly_eq(other) -> bool	True if the intervals overlap

4.3 Arithmetic

add, sub, mul, div, neg, recip, abs_val, pow_int(k), and sqrt() all return a new Interval that conservatively bounds the true operation. mul() takes the min/max of the four corner products; pow_int() special-cases even exponents when the interval straddles zero; recip() and div() raise ZeroDivisionError if the divisor interval contains zero; sqrt() requires lo >= 0 and uses a Newton iteration over Fraction. Python operators +, -, *, / are overloaded and accept int/Fraction on the right-hand side (auto-wrapped via Interval.point).

4.4 Module-level helper

```
interval_eval(x: ExactReal, prec: int = 40) -> Interval
```

Convenience wrapper equivalent to Interval.from_real(x, prec).

4.5 Example

```
from fractions import Fraction
from etc import ExactReal, Interval, interval_eval

x = ExactReal.pi()
iv = interval_eval(x, prec=64)
print(iv)                    # [3.1415927, 3.1415927]
print(iv.contains_zero())    # False

a = Interval.from_rationals(Fraction(1, 3), Fraction(1, 2))
b = Interval.point(Fraction(2))
print((a * b).width())       # exact Fraction width of the product interval
```

5. etc.core.complex -- ExactComplex

ExactComplex represents an exact complex number z = re + im*i where re and im are each ExactReal instances, so z is backed by two independent lazy Cauchy sequences over Q. Useful for Riemann surfaces, conformal maps, and algebraic curves over C.

5.1 Constructors

Constructor	Description
ExactComplex(re, im)	From two ExactReal parts
ExactComplex.from_real(x)	x + 0i
ExactComplex.from_rationals(a, b)	a + bi for rational a, b
ExactComplex.zero()	0 + 0i
ExactComplex.one()	1 + 0i
ExactComplex.i()	0 + 1i
ExactComplex.from_polar(r, theta)	r * e^(i*theta) = r*cos(theta) + r*sin(theta)*i

5.2 Evaluation, arithmetic and transcendental functions

Method	Returns	Notes
eval(prec)	(Fraction, Fraction)	(re_approx, im_approx), each within 2^-prec
neg()	ExactComplex	-z
conj()	ExactComplex	re - im*i
add / sub / mul(other)	ExactComplex	Standard complex arithmetic
abs_sq()	ExactReal	|z|^2 = re^2 + im^2
abs_val()	ExactReal	|z| = sqrt(re^2 + im^2)
recip()	ExactComplex	1/z = conj(z) / |z|^2
div(other)	ExactComplex	self * other.recip()
pow_int(k)	ExactComplex	self^k via binary exponentiation; negative k uses recip()
arg()	ExactReal	Principal argument in (-pi, pi], via arg(z) = 2*arctan(im / (|z| + re))
exp()	ExactComplex	e^z = e^re * (cos(im) + i*sin(im))
log()	ExactComplex	Principal value: ln|z| + i*arg(z)
sqrt()	ExactComplex	Principal square root: sqrt(|z|) * e^(i*arg(z)/2)
sin() / cos()	ExactComplex	Via sin/cos(re) and cosh/sinh(im) built from exp()

5.3 Python operator overloads and display

+, -, *, / are overloaded and accept a bare ExactReal on the right-hand side (auto-wrapped via ExactComplex.from_real). repr(z) renders the 53-bit float approximation as e.g. `ExactComplex(1.5 + 2i)`.

5.4 Example

```
from etc import ExactReal, ExactComplex

z = ExactComplex.from_rationals(1, 1)          # 1 + i
w = z.mul(z)                                    # i^2 term included: (1+i)^2 = 2i
print(w.eval(30))                               # (Fraction, Fraction) close to (0, 2)

r = ExactReal.from_rational(2)
theta = ExactReal.pi().div(ExactReal.from_rational(4))
polar = ExactComplex.from_polar(r, theta)       # 2 * e^{i*pi/4}
print(polar.abs_val().eval(20))                 # ~2
```

6. etc.core.integer -- ExactInteger

ExactInteger wraps Python's native arbitrary-precision int in a uniform ETC API and adds Euclidean/extended GCD, deterministic-then-probabilistic primality testing (Miller-Rabin, deterministic for n < 3.3x10^24), prime factorisation (trial division + Pollard-rho), modular arithmetic, integer roots, and combinatorics. All arithmetic is exact; no floating-point is used internally except in root-finding seed estimates that are subsequently corrected to exactness.

6.1 Constructors and conversion

Constructor / Method	Description
ExactInteger(value)	From a Python int (or anything int() accepts)
ExactInteger.zero() / .one()	The constants 0 and 1
ExactInteger.from_int(n)	Alias for the constructor
.value	Property returning the underlying Python int
.to_fraction()	Fraction(n, 1)
.to_real()	Convert to ExactReal (constant Cauchy sequence)

6.2 Arithmetic

Method	Description
add / sub / mul(other)	Standard integer arithmetic
floordiv(other) / mod(other)	Floor division and modulo; raise ZeroDivisionError on other=0
pow_int(k)	self^k for k >= 0 (negative k: use to_real().recip())
neg() / abs_val()	-self / |self|
gcd(other) / lcm(other)	Euclidean GCD / LCM
extended_gcd(other)	(g, s, t) with g = gcd(self, other) = s*self + t*other

6.3 Number theory

Method	Description
is_prime() -> bool	Deterministic Miller-Rabin for n < 3.3x10^24; 20-round probabilistic fallback above that
factorise() -> Dict[int, int]	Prime factorisation {prime: exponent}, via trial division then Pollard-rho
euler_totient() -> ExactInteger	phi(n) = n * prod_(p|n) (1 - 1/p)
moebius() -> int	Moebius function mu(n) in {-1, 0, 1}
modinv(m) -> ExactInteger	Modular inverse of self mod m; raises ValueError if gcd(self, m) != 1
modpow(exp, mod) -> ExactInteger	self^exp mod mod
jacobi(m) -> int	Jacobi symbol (self / m) for odd positive m
integer_sqrt() -> (ExactInteger, bool)	(floor(sqrt(n)), exact) where exact = True iff n is a perfect square
integer_nth_root(n) -> (ExactInteger, bool)	(floor(self^(1/n)), exact), via seeded Newton iteration

6.4 Combinatorics and digits

Method	Description
factorial() -> ExactInteger	self! for self >= 0
binomial(k) -> ExactInteger	C(self, k); returns 0 if k < 0 or k > self
ExactInteger.fibonacci(n)	Static method. n-th Fibonacci number via fast doubling, O(log n) multiplications
ExactInteger.catalan(n)	Static method. n-th Catalan number C(2n,n)/(n+1)
digits(base=10) -> List[int]	Digits of |self| in the given base, least-significant first
digit_sum(base=10) -> ExactInteger	Sum of digits of |self| in the given base

6.5 Module-level number-theory functions

The same algorithms are also exposed as free functions for use without constructing an ExactInteger: `is_prime(n)`, `factorise(n)`, `primes_up_to(n)` (Sieve of Eratosthenes), `nth_prime(n)`, `euler_totient(n)`, `moebius(n)`, `crt(residues, moduli)` (Chinese Remainder Theorem, requires pairwise-coprime moduli), `modinv(a, m)`, `jacobi_symbol(a, n)`, and `integer_nth_root(x, n)`.

6.6 Python operator overloads

+, -, *, //, %, ** (integer exponent only), unary -, abs(), ==, <, <=, hash(), int(), float(), repr()/str() are all supported; the binary operators accept a bare Python int on either side.

6.7 Example

```
from etc import ExactInteger

n = ExactInteger(9007199254740993)   # 2^53 + 1
print(n.is_prime())                   # False
print(n.factorise())                  # {6361: 1, 69431: 1, 20394401: 1}

f13 = ExactInteger.fibonacci(13)
print(f13)                            # 233

print(ExactInteger(100).binomial(ExactInteger(50)))   # C(100, 50), exact
```

7. etc.certified -- Certified[V]

Certified[V] is a generic typed wrapper pairing a computed value with its proof certificate. Every discrete output in ETC (sign, winding_number, orientation) is a Certified object.

Constructor

```
Certified(value: V, proof: Certificate)
```

Attributes and methods

Attribute/Method	Type	Description
.value	V	The computed result (e.g. int for sign or winding_number)
.proof	Certificate	Machine-checkable evidence. Use .proof.summary() to inspect.
.is_valid() -> bool	bool	Re-run the proof check from stored evidence
== other	bool	Compares .value only (not the proof)
int(cert)	int	Converts .value to int (useful for winding numbers)

Example

```
from etc import sign, ExactReal

result = sign(ExactReal.pi().sub(ExactReal.from_rational(3)))
print(result)             # Certified(value=1, claim='sign(x) = +1', VALID)
print(result.value)       # 1
print(int(result))        # 1
print(result == 1)        # True (compares value only)
```

8. etc.verify.certificate -- Certificate types

Every ETC computation producing a discrete result generates a Certificate. Certificates are self-contained, serialisable to JSON, and re-checkable independently of the original computation.

8.1 Base Certificate

Fields:

Field	Type	Description
claim	str	Human-readable statement of what is proved
method	str	Proof technique (cauchy_evaluation, symbolic, spot_check, etc.)
valid	bool	True iff all checks passed at creation time
evidence	dict	Serialisable dictionary of proof evidence
timestamp	float	Unix time when certificate was created

Methods:

Method	Description
is_valid() -> bool	Re-run checks from stored evidence
summary() -> str	Human-readable multi-line summary
to_json() -> str	Serialise to JSON string

8.2 Certificate subtypes

Subtype	Description
PointCertificate	Proves P in Space. evidence: coordinates, prec, space_name
PathCertificate	Proves gamma: [0,1] -> Space. evidence: n_spot_checks, params checked
HomotopyCertificate	Proves H: [0,1]^2 -> Space with endpoint conditions
IdentityCertificate	Proves f = 0 as a symbolic identity via SymPy trigsimp/simplify
ComposedCertificate	Conjunction of sub-certificates; is_valid() returns AND of all sub-checks

8.3 Factory functions

Function	Returns	Description
certify_point(point, space, prec=40)	PointCertificate	Check point in space via predicate at prec bits
certify_path(path, prec=40, n_spot=10)	PathCertificate	Spot-check path at n_spot rational parameter values
certify_homotopy(homotopy, prec=40, n_spot=5)	HomotopyCertificate	Spot-check H on n_spot x n_spot grid
certify_identity(lhs, rhs, variables=None)	IdentityCertificate	SymPy simplify(lhs - rhs) == 0
compose(*certs)	ComposedCertificate	Conjoin multiple certificates

8.4 CertificateStore

A named dictionary of certificates for batch management:

```
from etc.verify.certificate import CertificateStore, certify_identity
import sympy as sp

x = sp.Symbol('x')
store = CertificateStore()
store.add('pythagorean', certify_identity(sp.cos(x)**2 + sp.sin(x)**2, 1, [x]))
print(store.all_valid())   # True
print(len(store))          # 1
cert = store.get('pythagorean')
```

9. etc.verify.symbolic -- SymPy verification bridge

This module uses SymPy only as a verification oracle: ETC never trusts SymPy to compute the answer, only to certify that an answer already found by exact arithmetic is correct. SymPy is imported lazily; calling any function in this module without SymPy installed raises ImportError with a message pointing to `pip install sympy`.

9.1 Conversion utilities

Function	Description
real_to_sympy(x, prec=60)	Convert an ExactReal to a sympy.Rational approximant within 2^-prec
sympy_to_real(expr, free_vars=None)	Convert a closed-form SymPy expression to an ExactReal. Recognises sp.pi, sp.E, rationals, +, *, and rational powers exactly; falls back to 20-digit evalf() otherwise. free_vars is an optional {symbol: ExactReal} substitution map.

9.2 Core verifiers

Function	Description
verify_on_curve(point, curve, prec=60)	Certify a (ExactReal, ExactReal) point lies on an AlgebraicCurve: numerical check (|f| < 2^-(prec/2)) AND symbolic simplify(f) == 0
verify_path_in_space(path_exprs, param_sym, space, n_spot=11, prec=40)	Symbolic membership proof via Space.verify_path_symbolic, plus n_spot numerical spot-checks at rational parameter values
verify_homotopy(homotopy, n_grid=7)	Three-layer check: endpoint conditions, grid spot-check, and (if available) symbolic verification
verify_identity(lhs, rhs, variables=None, assumptions=None, prec=40)	simplify(expand(lhs - rhs)) == 0, retrying with trigsimp / radsimp / powsimp if the first pass fails
check_polynomial_identity(poly, subs_exact, prec=60)	Substitute exact ExactReal values into a SymPy polynomial and confirm the result is (numerically and/or symbolically) zero
simplify_predicate(expr, assumptions=None)	Return a simplified SymPy expression, optionally under given assumptions

9.3 Derivative and ideal-membership checks

Function	Description
diff_check(f_sympy, x_sym, f_exact, x_val, prec=40, order=1)	Confirm the SymPy derivative of f matches the finite-difference derivative from etc.analysis.calculus.Calculus, within 2^-(prec/3)
in_ideal(poly, generators, variables, domain='ZZ')	Groebner-basis membership test: True if poly lies in the ideal generated by generators

9.4 High-level object verification

Function	Description
verify_point(point, space, prec=60)	Verify a Point (or raw Coords tuple) lies in a Space using both the exact predicate and, if available, symbolic substitution into space.sympy_pred
verify_curve_parametrisation(curve, param_curve, n_spot=11, prec=40)	Verify a ParametricCurve lies on an AlgebraicCurve at n_spot sampled parameter values

10. etc.verify.lean_syntax -- SymPy-to-Lean 4 syntax

Translates SymPy expressions into syntactically valid Lean 4 / Mathlib4 source text. This module does not invoke the Lean compiler; it only guarantees syntactic translation for the operators and functions in its table below. It is not a general-purpose SymPy-to-Lean elaborator: expressions using functions outside the table fall back to a clearly marked `/- UNSUPPORTED SYMPY NODE -/` comment rather than emitting a guess that might silently be wrong.

10.1 Function table

SymPy function	Lean 4 / Mathlib rendering
sp.sin	Real.sin
sp.cos	Real.cos
sp.tan	Real.tan
sp.exp	Real.exp
sp.log	Real.log
sp.sqrt	Real.sqrt
sp.Abs	abs

Each of these renders as prefix juxtaposition (`Real.sin x`, not `sin(x)`), parenthesising the argument only when it is not itself an atom (a Symbol or Number). sp.Pow renders as `base ^ exp`, except that an exponent of exactly 1/2 is rendered as `Real.sqrt base`. Integers, rationals and floats are rendered with explicit `: R` type ascriptions where needed so the resulting fragment type-checks as a real-number expression.

10.2 Public functions

Function	Description
sympy_to_lean(expr) -> str	Translate a SymPy expression to a Lean 4 source-text fragment
free_variables_lean(variables) -> List[str]	Render a list of SymPy symbols as Lean 4 identifier strings
lean_forall_theorem(name, lhs, rhs, variables, tactic='ring') -> str	Render 'theorem name : forall v1 v2 ... : R, lhs = rhs := by tactic', binding every free variable explicitly so the theorem is closed and parses

Note: `lean_forall_theorem` guarantees the generated theorem statement parses in Lean 4 (no unbound identifiers); whether the given `tactic` actually closes the goal is a separate question the caller must still verify against the real Lean compiler.

10.3 Example

```
import sympy as sp
from etc.verify.lean_syntax import sympy_to_lean, lean_forall_theorem

x = sp.Symbol('x')
expr = sp.cos(x)**2 + sp.sin(x)**2
print(sympy_to_lean(expr))
# Real.cos x ^ 2 + Real.sin x ^ 2

print(lean_forall_theorem('pythagorean', expr, sp.Integer(1), [x]))
# theorem pythagorean : forall x : R, Real.cos x ^ 2 + Real.sin x ^ 2 = 1 := by
#   ring
```

11. etc.verify.formal -- Lean 4 Export

The Lean4Exporter translates ETC certificates into Lean 4 source code targeting Mathlib4, using etc.verify.lean_syntax (Section 10) to render SymPy expressions -- not raw Python str() -- so that generated identity theorems parse correctly. Algebraic identities provable by ring or norm_num are fully proved. Transcendental or continuity properties receive sorry stubs with explicit OPEN OBLIGATION labels and proof strategies.

Lean4Exporter

```
exporter = Lean4Exporter()
exporter.export_point_certificate(cert, name='myPoint')
exporter.export_path_certificate(cert, name='myPath')
exporter.export_homotopy_certificate(cert, name='myHomotopy')
exporter.export_identity_certificate(cert, name='pythagorean')
exporter.export_winding_number_certificate(cert, name='myWinding')
exporter.export_composed_certificate(cert, name='myComposed')
exporter.export(cert, name='auto')      # dispatch on certificate type

lean_src = exporter.render()            # str: Lean 4 source code
exporter.write('output.lean')           # write to file
```

Each export_* method appends a section to an internal buffer and returns the unique Lean identifier chosen for that declaration; render() concatenates the fixed Mathlib import header with all buffered sections.

certificate_to_lean() convenience function

```
from etc.verify.formal import certificate_to_lean
from etc.verify.certificate import certify_identity
import sympy as sp

x = sp.Symbol('x')
cert = certify_identity(sp.cos(x)**2 + sp.sin(x)**2, sp.Integer(1), [x])
src = certificate_to_lean(cert, name='pythagorean')
print(src)
# import Mathlib
# theorem pythagorean : ...
#   ring
```

Open proof obligations

Four proof obligations are marked sorry in the generated output. Each sorry carries an OPEN OBLIGATION comment with the precise statement and a suggested proof strategy:

Obligation	Strategy
Path membership continuity: forall t in [0,1], gamma(t) in Space	Lean ContinuousOn tactics with interval-arithmetic bounds
Winding number arctan2 bound	Mathlib Real.arctan lemmas with Arb interval certificates
Homotopy endpoint conditions: H(0,t) = gamma_0(t) and H(1,t) = gamma_1(t)	Density arguments at rational parameters; or ring/norm_num if symbolic
Pythagorean identity	exact Real.cos_sq_add_sin_sq x (one-line Mathlib proof)

For an identity certificate whose difference is exactly zero but whose rendered Lean text uses a transcendental function (Real.sin, Real.cos, Real.tan, Real.exp, Real.log, or Real.sqrt), the exporter substitutes a sorry with a pointer to the relevant Mathlib lemma rather than emitting `ring`, since the ring tactic cannot close goals involving those functions.

CI status. Generated .lean files are not checked against the Lean 4 kernel in the package's CI (no Lean toolchain is assumed to be installed); run `lake build` yourself to confirm before relying on the output as a machine-checked proof.

12. etc.topology -- Space, Path, Homotopy

12.1 Space

```
Space(
    predicate: Callable[[Coords], bool],
    dim: int,
    sympy_pred = None,     # sympy.Expr: f(coords) = 0 on Space
    sympy_coords= None,    # Tuple[sympy.Symbol, ...]
    name: str = 'Space',
    description: str = '',
)
```

Method	Description
contains(coords: Coords) -> bool	Check membership via exact predicate
verify_path_symbolic(path_exprs, param_sym) -> bool	Symbolically prove path stays in Space for all parameter values

Factory functions for common spaces:

Function	Description
make_Lp_circle(p: float) -> Space	L^p 'circle' {(x,y) | |x|^p + |y|^p = 1} in R^2
make_sphere(r=1) -> Space	Unit sphere in R^3
make_torus(R, r) -> Space	Torus with major radius R and minor radius r

12.2 UnitInterval

UnitInterval wraps an ExactReal constrained to [0, 1]. Construction raises ValueError if the value is outside [0, 1].

```
UnitInterval(x: ExactReal, _skip_check=False)
UnitInterval.from_rational(p, q=1) -> UnitInterval
UnitInterval.zero() -> UnitInterval    # t = 0
UnitInterval.one()  -> UnitInterval    # t = 1
UnitInterval.half() -> UnitInterval    # t = 1/2
t.complement()       -> UnitInterval    # 1 - t
```

12.3 Path

```
Path(
    path_fn: Callable[[UnitInterval], Coords],
    space: Space,
    sympy_path = None,   # Tuple[sympy.Expr, ...] in param_sym
    sympy_param = None,  # sympy.Symbol
    name: str = 'gamma',
)
```

Method	Description
evaluate(t: UnitInterval) -> Coords	Evaluate path at parameter t
spot_check(n=10, prec=40) -> bool	Check membership at n equally-spaced rational parameter values
symbolic_verify() -> bool	Symbolically prove path stays in Space (requires sympy_path and sympy_param)
is_closed(prec=40) -> bool	Check gamma(0) == gamma(1) within 2^-prec

12.4 Homotopy

```
Homotopy(
    homotopy_fn: Callable[[UnitInterval, UnitInterval], Coords],
    space: Space,
    path0: Path,   # H(0, .) = path0
    path1: Path,   # H(1, .) = path1
    sympy_homotopy: Optional[Tuple] = None,
    s_sym = None,
    t_sym = None,
    name: str = 'H',
)
```

Method	Description
spot_check(n=5, prec=40) -> bool	Check H on n x n parameter grid
verify_endpoints(prec=40) -> bool	Check H(0,t) = path0(t) and H(1,t) = path1(t) at spot points
symbolic_verify() -> bool	Full algebraic proof via SymPy
verify() -> bool	Run all three verification steps

13. etc.topology.manifold -- Manifold, Chart, Atlas

A Manifold is a topological Space equipped with an Atlas of Charts. Each Chart is a homeomorphism between an open subset of the manifold and an open subset of R^n (the coordinate domain). The Atlas certifies that the manifold is locally Euclidean: every sample point is contained in at least one chart, and transition maps on chart overlaps are checked numerically (and symbolically, where SymPy expressions are supplied). The implementation is constructive: membership, coordinates, and transitions are all computable, not merely postulated.

13.1 Chart

```
Chart(
    forward:            Callable[[Coords], Coords],   # manifold -> local R^n
    inverse:            Callable[[Coords], Coords],   # local R^n -> manifold
    domain_predicate:   Callable[[Coords], bool],      # True if pt in U
    dim:                int,
    name:               str = "chart",
    sympy_forward=None, sympy_inverse=None,
    sympy_ambient_coords=None, sympy_local_coords=None,
)
```

Method	Description
contains(pt) -> bool	True if pt lies in this chart's domain U
to_local(pt) -> Coords	Map a manifold point to local coordinates; raises ValueError if pt not in domain
from_local(local) -> Coords	Map local coordinates back to the ambient manifold
transition_to(other, test_points=None, prec=30) -> bool	Verify the transition map is well-defined on the overlap by round-tripping at test_points; adds a symbolic check when both charts carry SymPy data

13.2 Atlas

Method	Description
add_chart(chart)	Append a Chart to the atlas
covers(pt) -> bool	True if some chart contains pt
chart_for(pt) -> Optional[Chart]	Return the first chart containing pt, or None
verify_coverage(sample_points, prec=30) -> bool	Check every sample point is covered by at least one chart
verify_transitions(sample_points, prec=30) -> bool	Check transition maps are consistent on all pairwise chart overlaps

13.3 Manifold

Method	Description
contains(pt) -> bool	Membership via the underlying Space predicate
local_coords(pt) -> (Chart, Coords)	Return the covering chart and pt's local coordinates; raises ValueError if uncovered
verify(sample_points, prec=30) -> bool	Full atlas verification: space membership, chart coverage, and transition consistency
tangent_basis(pt, chart, prec=30) -> List[Tuple[float,...]]	Approximate tangent frame at pt via finite differences on the chart's inverse map

13.4 Factory functions

Function	Description
make_circle_atlas() -> Manifold	S^1 with two stereographic charts (chart N excludes y=1, chart S excludes y=-1)
make_sphere_atlas() -> Manifold	S^2 with two stereographic charts (poles removed)
make_torus_atlas(R, r) -> Manifold	T^2 with a single angle-product chart (theta, phi), valid away from the meridian x=0, y<=0

13.5 Example

```
from etc.topology.manifold import make_sphere_atlas
from etc.core.real import ExactReal

sphere = make_sphere_atlas()
zero, one = ExactReal.zero(), ExactReal.one()
pt = (zero, zero, one)                # north pole
chart, local = sphere.local_coords(pt) # chart S covers it (chart N excludes the pole)
print(chart.name)                      # 'stereo_S'
print(sphere.verify([pt]))             # True
```

14. etc.topology.invariants -- winding_number()

```
winding_number(
    path: Path,
    base_point: Optional[Tuple[ExactReal, ExactReal]] = None,
    prec: int = 53,
    n_steps: int = 200,
) -> Certified[int]
```

Parameters

Parameter	Type	Description
path	Path	A closed path in R^2 (space must be 2-dimensional)
base_point	(ExactReal, ExactReal) or None	The point to wind around. Defaults to (0, 0) if None.
prec	int	Working precision in bits for angle computations (default 53)
n_steps	int	Number of angle samples. Higher values reduce rounding error but increase runtime. Default 200; paper uses 400.

Returns

Certified[int] with .value = winding number in Z. The .proof Certificate includes rounding_error (accumulated arctan2 error) and path_closed_gap (|gamma(0) - gamma(1)|).

Raises

ValueError if the path passes through the base point (angle undefined), or if the path is not closed (gamma(0) != gamma(1) within tolerance).

Example

```
import math
from fractions import Fraction
from etc import ExactReal, winding_number
from etc.topology.space import make_Lp_circle
from etc.topology.path import Path, UnitInterval

# Build a circular path winding once around the origin
space = make_Lp_circle(2)

def gamma(t: UnitInterval):
    theta = float(t.value.eval(20)) * 2 * math.pi
    return (
        ExactReal.from_rational(Fraction(math.cos(theta)).limit_denominator(10**9)),
        ExactReal.from_rational(Fraction(math.sin(theta)).limit_denominator(10**9)),
    )

path = Path(gamma, space, name='circle_1x')
result = winding_number(path, n_steps=400)
print(result.value)                                # 1
print(result.is_valid())                           # True
print(result.proof.evidence['rounding_error'])     # ~4.44e-16
```

15. etc.geometry -- Curves, Polygon, Surfaces

15.1 AlgebraicCurve

```
AlgebraicCurve(sympy_poly, x_sym, y_sym, name='curve')

# Methods
.as_space(check_prec=40) -> Space              # build Space from polynomial predicate
.point(x, y) -> Point                          # construct and certify a point on the curve
.tangent_vector(pt, prec=30) -> (float, float) # approximate tangent direction
```

15.2 ParametricCurve

```
ParametricCurve(
    x_fn: Callable[[ExactReal], ExactReal],
    y_fn: Callable[[ExactReal], ExactReal],
    name: str = 'curve',
)

.evaluate(t: ExactReal) -> (ExactReal, ExactReal)
.as_path(space, name) -> Path
```

15.3 ExactPolygon

```
ExactPolygon(vertices: List[Tuple[ExactReal, ExactReal]])
# Requires at least 3 vertices

.n_vertices() -> int
.signed_area(prec=60) -> ExactReal   # exact via Shoelace formula
.area(prec=60) -> ExactReal
.centroid(prec=60) -> (ExactReal, ExactReal)
.is_convex(prec=40) -> bool
.winding_number(pt, prec=53) -> int  # exact integer
.contains_point(pt, prec=53) -> bool # True if winding != 0
```

15.4 AlgebraicSurface

```
AlgebraicSurface(sympy_poly, x_sym, y_sym, z_sym, name='surface')

.as_space(check_prec=40) -> Space
.point(x, y, z) -> Point
.normal_vector(pt, prec=30) -> (float, float, float)  # unit normal grad f / |grad f|
```

16. etc.analysis -- Series, Calculus, ODE

The analysis module provides power series, symbolic calculus, and ODE tools built on top of ExactReal and SymPy.

16.1 PowerSeries and TaylorSeries

```
from etc.analysis.series import PowerSeries, TaylorSeries

# PowerSeries: sum_{k=0}^{N} c_k * x^k
coeffs = [ExactReal.from_rational(c) for c in [1, 0, -1, 0, 1]]
ps = PowerSeries(coeffs)
val = ps.evaluate(ExactReal.from_rational(Fraction(1, 2)), prec=60)

# TaylorSeries: expand a SymPy expr around x=a to order n
import sympy as sp
x = sp.Symbol('x')
ts = TaylorSeries(sp.sin(x), x, center=0, order=10)
approx = ts.to_power_series()   # -> PowerSeries
```

16.2 Calculus

```
from etc.analysis.calculus import Calculus

calc = Calculus()

# Symbolic derivative
x = sp.Symbol('x')
df = calc.differentiate(sp.sin(x)**2, x)   # 2*sin(x)*cos(x)

# Numerical integration via rectangle rule on ExactReal grid
integrand = lambda t: t.mul(t)   # f(t) = t^2
result = calc.integrate(integrand, a=0, b=1, n_intervals=100, prec=40)
# result is ExactReal; float(result.eval(40)) ~ 0.3333...
```

16.3 ODE

```
from etc.analysis.ode import ODE

# Solve y' = f(t, y) with initial condition y(0) = y0
# using exact Euler method on rational grid
ode = ODE(
    f=lambda t, y: y,               # y' = y (solution: e^t)
    y0=ExactReal.from_rational(1),  # y(0) = 1
    t_span=(0, 1),
    n_steps=100,
    prec=60,
)
sol = ode.solve()   # -> ODESolution
print(float(sol.y_at(1).eval(40)))   # approx e ~ 2.71828...
```

17. etc.problems -- Problem registry

The problems module provides a structured interface for the canonical benchmark problems implemented in ETC. Each problem class implements solve() -> Certified and verify(answer) -> bool. To run all seven canonical failure cases and print a formatted comparison table matching Table 1 of the paper, use scripts/run_comparison.py; two of those seven are implemented as registered Problem subclasses (the remainder are computed inline by the comparison script).

Problem abstract base class

```
from etc.problems.base import Problem

class Problem(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def description(self) -> str: ...

    @abstractmethod
    def solve(self) -> Certified[Any]: ...

    @abstractmethod
    def verify(self, answer: Any) -> bool: ...
```

ProblemRegistry

```
from etc.problems.registry import registry

registry.register(RumpPolynomial())
registry.list()                       # prints all problem names
p = registry.get('rump_polynomial')   # fetch by name
result = p.solve()                    # Certified answer
print(p.verify(result.value))         # True
```

Canonical problems (canonical_problems module)

Class	Problem	Exact answer
RumpPolynomial	f(77617, 33096) -- Rump 1988	Fraction(-54767, 66192)
L5WindingNumber	Winding number of L^5 circle path around origin	1 (certified)

18. Timing API: timed_eval and benchmark_eval

ETC provides two timing interfaces following the Johnson (2002) methodology recommended by ACM TOMS.

18.1 timed_eval()

```
# Instance method on ExactReal
val, t = ExactReal.pi().timed_eval(128)
# val: Fraction -- exact rational at 128-bit precision
# t:   float    -- wall-clock seconds (0.0 on cache hit)

# Example output:
print(f"pi ~ {float(val):.15f} [{t*1000:.2f} ms]")
# pi ~ 3.141592653589793 [0.39 ms]
```

18.2 benchmark_eval()

```
from etc import ExactReal, benchmark_eval

# benchmark_eval(x, precisions, repeat=3) -> List[(int, float)]
# Creates a FRESH ExactReal instance (empty cache) per precision.
# Returns the MINIMUM wall-clock time over 'repeat' runs.
results = benchmark_eval(
    ExactReal.pi(),
    precisions=[8, 16, 32, 64, 128, 256, 512, 1024],
    repeat=3,
)
for n, t in results:
    print(f"n={n:5d} {t*1000:.3f} ms")
```

Note on the pre-asymptotic regime

At precision levels n <= 1024 bits, the measured scaling exponents are below the theoretical O(n^2 log n) value of 2.0 because Python interpreter overhead and the constant-factor cost of Fraction GCD dominate. Reliable asymptotic exponent fitting requires n >= 4096. The benchmark script (scripts/benchmark.py) reports fitted exponents for each doubling.

19. Error handling and known limitations

19.1 Exceptions

Exception	Raised when
ValueError	precision < 0 in eval() or timed_eval(); ExactReal argument out of domain (e.g. log of negative, sqrt of negative); path not closed in winding_number(); base_point on path in winding_number(); Interval endpoints with lo > hi; Interval.sqrt() on a negative lower bound; a point requested outside every Chart's domain in a Manifold
ZeroDivisionError	recip() or div() called when denominator is zero or converges to zero faster than the precision search can establish |x| > 0; Interval.recip()/div() when the divisor interval contains zero; ExactInteger.floordiv()/mod() with a zero divisor
ImportError	SymPy not installed when using etc.verify.symbolic or certify_identity()

19.2 Known limitations

Trigonometric argument reduction for large inputs. sin() and cos() reduce modulo 2*pi using round(float(x / 2*pi)), which introduces a float rounding error of up to 2^-52 relative to the true quotient. For inputs where x / 2*pi is within 2^-52 of a half-integer, the wrong multiple may be selected. In practice this is not triggered for |x| <= 10^3 and n <= 512 bits, but users should not rely on sin() or cos() for very large arguments (|x| >> 2^52) without independent range reduction.

from_sympy() uses mpmath internal API. ExactReal.from_sympy() accesses mpmath's _mpf_ internal attribute to convert symbolic expressions. This may break on future mpmath versions. For production use, prefer ExactReal.from_rational() with a pre-computed rational approximant.

recip() iteration limit. recip() searches up to k=300 to establish |self| > 0. For values that converge to zero very slowly this limit may be reached, raising ZeroDivisionError even for non-zero inputs. Increase the limit by subclassing ExactReal if needed.

Interval width growth under repeated multiplication. Interval arithmetic is conservative by construction: repeated mul() calls can widen an interval faster than the true error grows (dependency problem), so Interval should be treated as a fast pre-filter, not a tight error bound, for long arithmetic chains -- use ExactReal directly when a tight bound is required.

ExactInteger.integer_nth_root() seeding. The initial root estimate is seeded with a floating-point call (x ** (1.0 / n)) before being corrected to exactness by Newton iteration and a final bracketing pass; for extremely large x and n this float seed can occasionally require several extra correction iterations, though the returned (r, exact) pair is always exact.

Lean 4 sorry stubs. Four proof obligations remain as sorry stubs in Lean 4 export: path membership continuity, winding number arctan2 bounds, homotopy endpoint conditions universally, and the Pythagorean identity. Each sorry carries an OPEN OBLIGATION comment with the precise statement and a suggested proof strategy. See Section 11 for details.

Performance. Exact arithmetic over Fraction is substantially slower than float64: pi at 512 bits takes approximately 1.7 ms versus nanoseconds for a float division. ETC is suitable where correctness is paramount and the number of exact operations is moderate. It is not suitable for high-throughput numerical simulation or large-scale matrix operations.

20. Comparison script: run_comparison.py

The script scripts/run_comparison.py runs all seven canonical floating-point failure cases and prints a formatted Float64 vs ETC comparison table that directly reproduces Table 1 of the accompanying paper. All ETC results are computed live by running the library; no values are hardcoded in the script.

Usage

```
python scripts/run_comparison.py
```

Expected output

```
Float64 vs ETC -- Seven Canonical Floating-Point Failure Cases
============================================================
 # | Problem                      | float64 result         | ETC result
-------------------------------------------------------------------------------------------
 1 | Rump f(77617,33096)          | -1.180592e+21          | -54767/66192 = -0.8...
 2 | 10^16 + 1 - 10^16            | 0.0 (should be 1)      | 1 (exact integer)
 3 | (sqrt(x+1)-sqrt(x))*(...)-1  | 0.1780 (should be 0)   | |f| = 1.32e-85; pro...
 4 | (x-1)^10 at x=1+1e-6         | 4.66e-15 (s/b 1e-60)   | 1/10^60 (exact)
 5 | Orientation eps=1e-16        | cross=0.0 (collinear)  | sign=+1 CCW cert_va...
 6 | Winding number (L5 curve)    | 1.000000 (uncertified) | 1 (exact int, cert...
 7 | cos^2(x)+sin^2(x)=1          | max err=2.22e-16       | proved for all x in...

Summary: float64 passed 0/7 problems | ETC passed 7/7 problems
```

What each column means

Column	Description
#	Problem number matching Table 1 of the paper
Problem	Short label for the canonical failure case
float64 result	The value returned by Python float / NumPy float64. Includes a note on what the correct answer should be.
ETC result	The exact value computed by ETC, shown as a Fraction or symbolic proof result. cert_valid=True confirms the Certificate.is_valid() check passes.
Float	PASS if float64 gives the correct answer (certifiably); FAIL otherwise. All seven problems are FAIL for float64.
ETC	PASS if ETC gives the correct certified answer; FAIL otherwise. All seven problems are PASS for ETC.

What each problem tests

#	Failure mode tested
1	Catastrophic cancellation: terms reach 10^36, sum is O(1)
2	Absorption: 10^16 + 1 rounds to 10^16 in float64
3	Difference-of-squares identity: float loses all significant digits
4	Polynomial sensitivity near a root: wrong sign and magnitude
5	Orientation predicate: cross-product underflows to zero
6	Winding number: float cannot certify the integer result
7	Pythagorean identity: float cannot prove for all x in R

Runtime

The script takes approximately 10-30 seconds depending on hardware. Problem 6 (L5WindingNumber) is the slowest step, building a 200-vertex exact polygon and computing the winding number via ExactPolygon. No additional dependencies beyond the base install are required.

Relationship to the test suite

The test suite (pytest tests/ -v) verifies correctness via assertions. The comparison script is complementary: it prints human-readable output suitable for a reviewer to read directly, showing the float64 failure and ETC success side by side for each problem.

End of ETC User Manual. For the theoretical background and experimental results, see the accompanying ACM TOMS manuscript.